Gymnázium J. K. Tyla



Agentické inženýrství ve vývoji softwaru

Návrh a implementace DarkFactory

ODBORNÁ PRÁCE

Autor práce — Patrik Marius, 4.D

Vedoucí práce — Michal Dočekal

2026

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnáziu J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne _____

Podpis autora práce — _____

Anotace

Práce se zabývá současným agentickým inženýrstvím ve vývoji softwaru. S rostoucími schopnostmi a adopcí generativní AI se rozšiřují možnosti agentů pro vývoj softwaru, jejich praktické využití je však stále méně rozšířené než běžné používání chatbotů. Cílem práce je popsat a systematizovat principy současného agentického inženýrství a na systému DarkFactory ukázat, co jejich propojení umožňuje v praxi.

Teoretická část vysvětluje vztah mezi jazykovým modelem, agentem a agentní vrstvou a shrnuje práci s kontextem, nástroji, trvalým stavem, specifikací, orchestrací, programovým ověřováním a lidskými rozhodovacími body. Popisuje také současnou podobu softwarové továrny, v níž jsou agenti začleněni do běžného repozitářového procesu.

Praktická část má podobu inženýrské případové studie DarkFactory, záměrně jednoduché agentické softwarové továrny nativně postavené na GitHubu. Systém byl implementován pomocí komerčních agentů. Již v této podobě dokáže agenty sám spouštět a řídit a vytváří základ pro další rozvoj stejným procesem.

Studie ukazuje, že praktický agentický proces vzniká kombinací modelového úsudku s trvalým stavem mimo model, programovou orchestrací, deterministickými kontrolami a lidskými branami. Současně odhaluje omezení počáteční implementace, zejména u modelové revize, opakování verifikace a změn schváleného rozsahu. Přínosem práce je systematizace těchto principů a ukázka jejich konkrétní realizace v jednom produkčním vývojovém procesu.

Klíčová slova — coding agenti; harness; softwarová továrna; orchestrace; generativní AI

Abstract

The thesis addresses contemporary agentic engineering in software development. As the capabilities and adoption of generative AI grow, the possibilities of coding agents also expand, but their practical use remains less widespread than ordinary chatbot use. The aim of the thesis is to describe and systematize the principles of contemporary agentic engineering and, using DarkFactory, show what their combination enables in practice.

The theoretical part explains the relationship between the language model, the agent, and the harness layer and summarizes work with context, tools, durable state, specification, orchestration, programmatic verification, and human decision points. It also describes the contemporary form of the software factory, in which coding agents are integrated into ordinary repository workflows.

The practical part takes the form of an engineering case study of DarkFactory, a deliberately simple agentic software factory built natively on GitHub. The system was implemented using commercial coding agents. Even in this initial form, it can already launch and manage agents itself and creates a foundation for further development using the same process.

The study shows that a practical agentic workflow arises from combining model judgment with durable state outside the model, programmatic orchestration, deterministic checks, and human gates. At the same time, it reveals limitations of the initial implementation, particularly in model-based review, repeated verification, and changes to the approved scope. The contribution of the thesis is the systematization of these principles and a demonstration of their concrete implementation in a single production development process.

Keywords — coding agents; harness; software factory; orchestration; generative AI

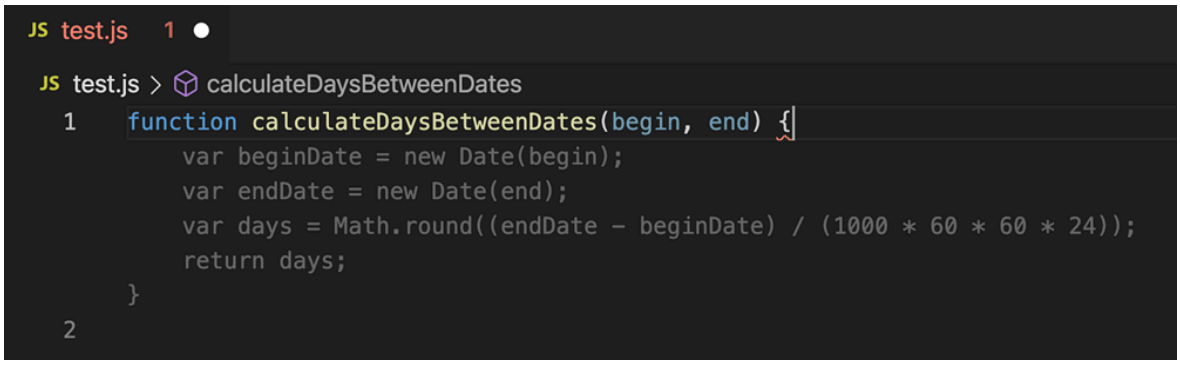
Obsah

Prohlášení 2
1 Úvod 6
1.1 Motivace – vývoj a adopce generativní AI 6
1.2 Cíl, výzkumná otázka a vymezení 9
1.3 Terminologie 10
2 Teoretická část 11
2.1 Agent – co to je a jak funguje 11
2.1.1 Large Language Model (jazykový model) 11
2.1.2 Agentní smyčka 12
2.1.3 Nástroje a rozšíření 13
2.1.4 Kontext 14
2.2 Agentické inženýrství 15
2.2.1 Specifikace 15
2.2.2 Orchestrace 17
2.3 Softwarová továrna 19
3 Praktická část 21
3.1 Metodika 21
3.2 Návrh systému DarkFactory 22
3.3 Architektura systému 23
3.4 Průchod požadavku systémem 24
3.5 Implementace pomocí agentů 26
4 Zjištění a diskuse 27
5 Závěr 28
Seznam zdrojů 29
Seznam součástí textu 33

1 Úvod

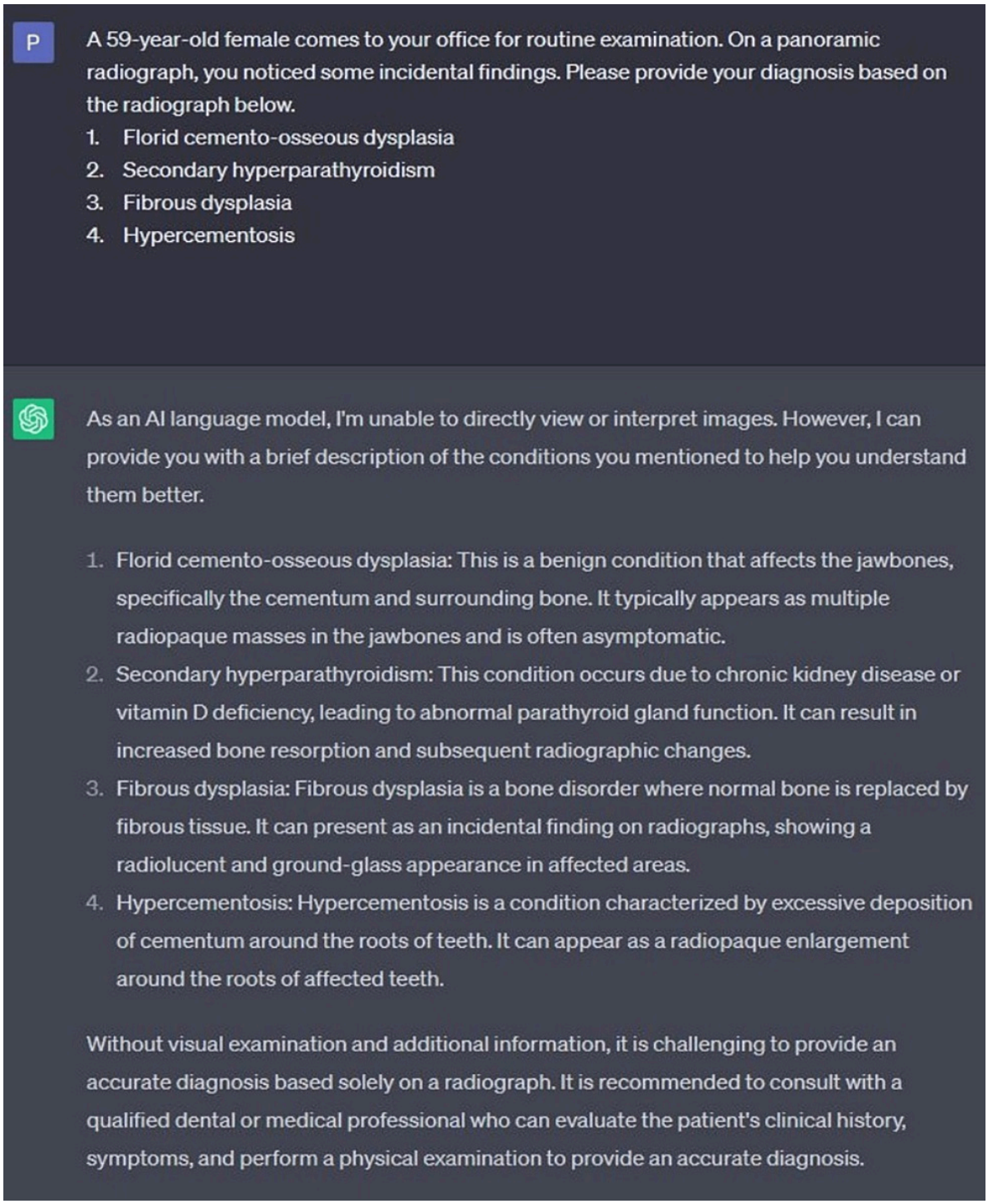
1.1 Motivace – vývoj a adopce generativní AI

Od svého vzniku se nástroje založené na jazykových modelech neustále zlepšují a roste také jejich adopce. Jednou z prvních široce používaných forem ve vývoji softwaru bylo doplňování kódu přímo v editoru (1), jehož příklad ukazuje Obrázek 1.



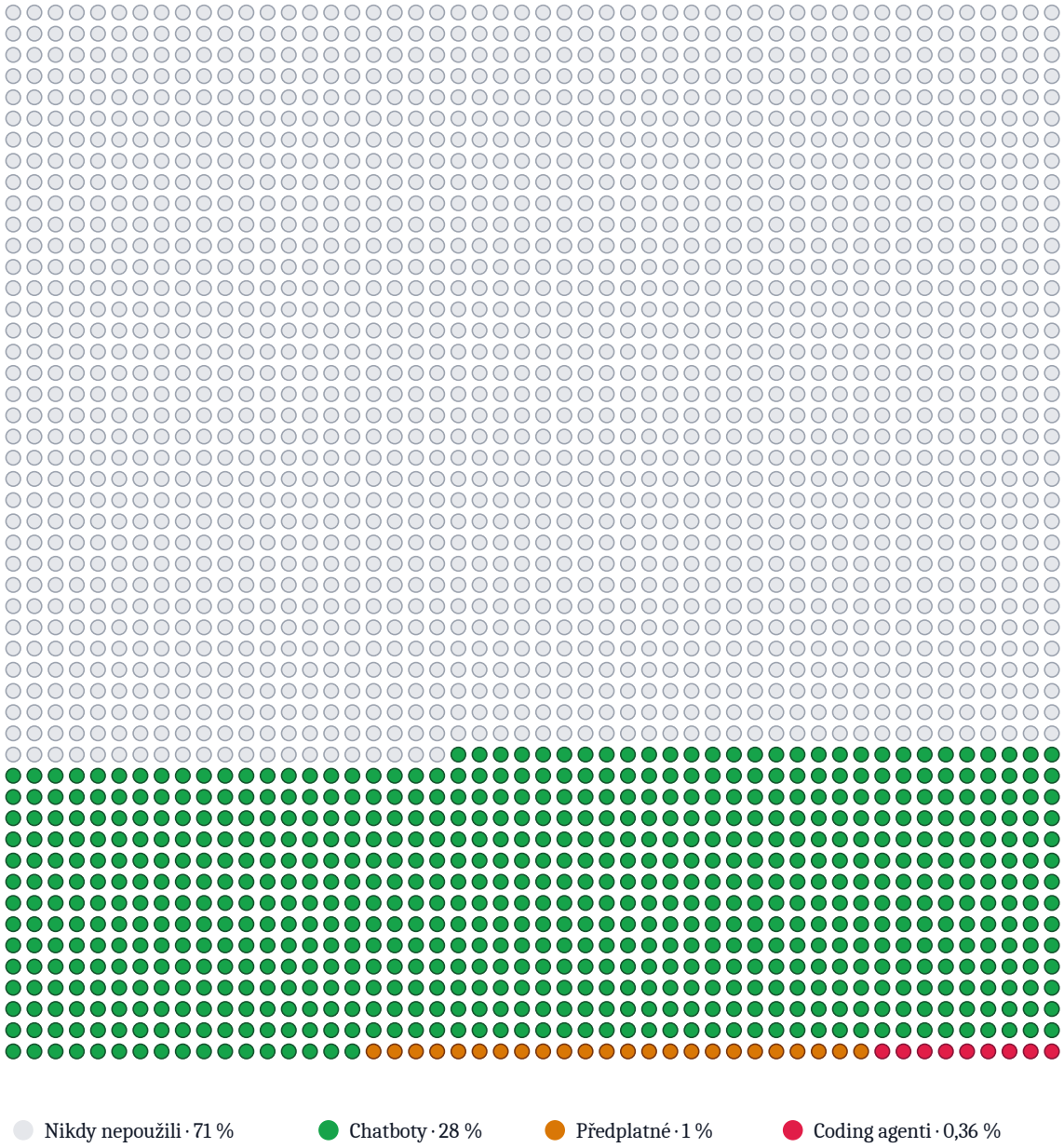
Obrázek 1 — Doplňování kódu v editoru (1).

Potom přišly konverzační chatboty, v nichž model sestavuje odpověď, ale nástroje mu zpravidla nebyly k dispozici, takže další práci stále prováděl uživatel. Tento způsob použití ilustruje Obrázek 2.



Obrázek 2 — Chatbot bez přístupu k obrázku (2).

Další posun představují ***Coding Agents (coding agenti)*** (3; 4; 5), kteří mohou získat přístup k souborům, příkazům a běhovému prostředí. Model tak již nejen navrhuje výsledek, ale může prostřednictvím nástrojů sám provádět jednotlivé kroky práce.



Obrázek 3 — Odhad využití generativní AI podle typu (6).

Podle jednoho zveřejněného odhadu používá bezplatné AI chatboty přibližně 28 % světové populace, zatímco pravidelní uživatelé těchto agentů tvoří přibližně 0,36 % (6) Obrázek 3. Agentické nástroje jsou tedy stále výrazně méně rozšířené než běžné konverzační použití generativní AI. Motivací této práce je proto ukázat, čeho lze s těmito nástroji dosáhnout při použití současných postupů agentického inženýrství.

1.2 Cíl, výzkumná otázka a vymezení

Cílem práce je popsat a systematizovat principy současného agentického inženýrství ve vývoji softwaru a na systému DarkFactory ukázat, co jejich propojení umožňuje v praxi. Pozornost je přitom věnována také způsobu, jakým lze agenty začlenit přímo do běžného repozitářového pracovního postupu. Takové propojení agentů s existujícími procesy vývoje se již používá v produkčním softwarovém inženýrství a představuje současnou podobu myšlenky softwarové továrny (7; 8).

Výzkumná otázka práce zní — *Jaké architektonické a procesní principy se opakují v současném agentickém vývoji softwaru a jak jsou realizovány v systému DarkFactory?*

Práce vychází z předpokladu, že jazykový model je v současných agentických systémech pouze jednou částí širšího celku. Praktické použití agentů proto závisí také na způsobu práce s kontextem, nástroji a trvalým stavem, na orchestraci jednotlivých kroků, automatickém ověřování a zapojení člověka do rozhodovacích bodů. Teoretická část tyto opakující se principy popisuje a praktická část ukazuje jejich propojení v jednom konkrétním systému.

Praktická část má podobu inženýrské případové studie DarkFactory, agentické softwarové továrny nativně postavené na GitHubu. Systém byl implementován pomocí komerčních agentů a je záměrně navržen jako jednoduchá počáteční implementace. Již v této podobě dokáže sama spouštět a řídit agenty a vytváří tak základ, který lze pomocí stejného procesu dále rozvíjet. Cílem případové studie není měřit obecnou úspěšnost jazykových modelů ani porovnávat jednotlivé agenty, ale popsat konkrétní realizaci současných postupů agentického inženýrství a ukázat, co jejich spojení umožňuje v praxi.

1.3 Terminologie

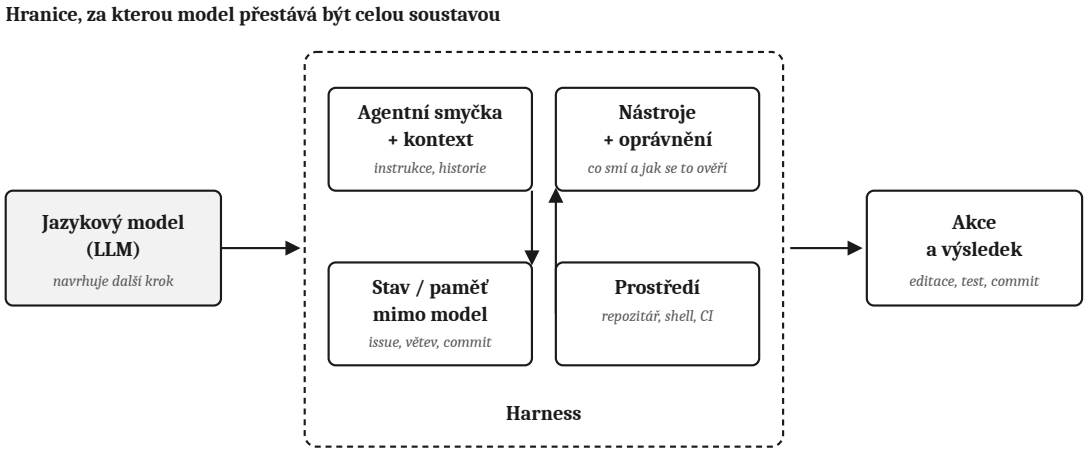
Anglické odborné termíny jsou v práci uváděny anglicky a při vhodném českém ekvivalentu také česky v závorce. Při prvním zavedení jsou označeny hvězdičkou, tučně a kurzívou. Stručná definice se uvádí v poznámce pod čarou, aby hlavní text mohl pokračovat rovnou mechanismem nebo použitím daného pojmu.

Rozlišení slov ***Agent*** a ***Agentic*** je zachováno i v češtině. Agent označuje samotného jednajícího, zatímco agentic popisuje schopnost systému jednat (9). Proto se používá například agentní smyčka, ale agentické inženýrství.

2 Teoretická část

2.1 Agent – co to je a jak funguje

U *__Agent__[1] se rozhodování a provedení dělí mezi model a *__Harness__[2] (10). Model navrhuje další krok, zatímco harness jej převádí na akci a vrací výsledek zpět do dalšího kroku. Rozvrstvení shrnuje Obrázek 4.



Obrázek 4 — Vrstvy agentického systému.

2.1.1 Large Language Model (jazykový model)

*__Large Language Model (jazykový model)__[3] v agentovi vytváří další výstup podle aktuálního kontextu. Současné modely běžně používají architekturu *__Transformer__[4] představenou v roce 2017 (11).

U *__Attention (pozornost)__[5] je prakticky důležité, že každý token může využít informace z ostatních pozic. U standardní plné pozornosti proto rostou výpočetní náklady přibližně s druhou mocninou délky kontextu.

Model sám mezi jednotlivými voláními neudržuje pracovní stav a bez okolního systému nemá přístup k souborům, příkazům ani nástrojům. Při inferenci zpracuje aktuální kontext a vytváří výstupní tokeny. Trvalý stav, nástroje a oprávnění proto musí dodat okolní harness.

[1] __Agent__ — Systém, v němž model prostřednictvím nástrojů jedná nad prostředím.

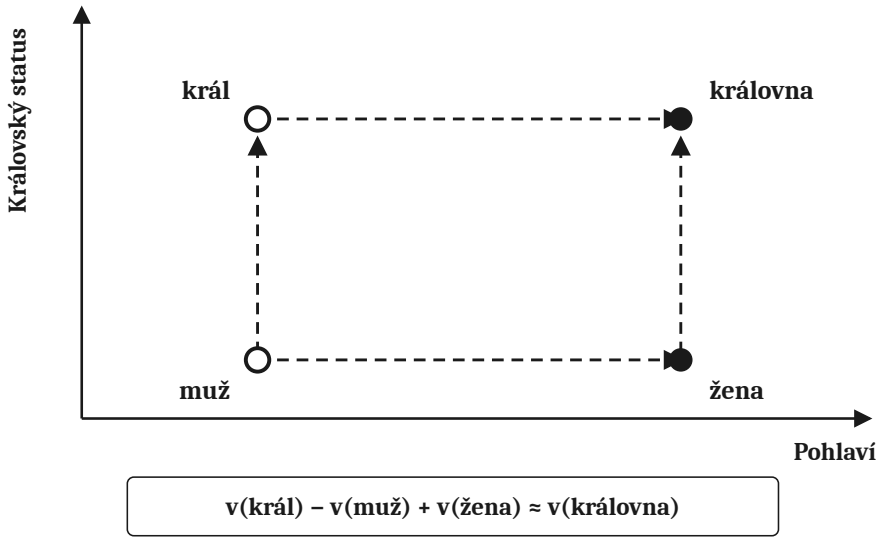
[2] __Harness__ — Vrstva kolem modelu, která mu předává kontext, nástroje, oprávnění, stav a pravidla běhu.

[3] __Large Language Model__ — Model, který z kontextu odhaduje pravděpodobnosti následujících tokenů.

[4] __Transformer__ — Neuronová architektura založená na mechanismu pozornosti, která zpracovává vztahy mezi tokeny v kontextu.

[5] __Attention__ — Mechanismus, který při výpočtu reprezentace tokenu váží informace z dalších tokenů v kontextu.

U ***Embedding (vektorová reprezentace)***[6] lze sémantické vztahy ukázat geometricky. Známým příkladem je vztah mezi slovy král, královna, muž a žena (12) Obrázek 5.



Obrázek 5 — Sémantické vztahy ve vektorovém prostoru (12).

2.1.2 Agentní smyčka

Agent Loop (agentní smyčka)[7] se zpravidla řídí vzorem ***ReAct (Reasoning and Acting)***[8] (13).

Jeden cyklus lze zjednodušit do pěti kroků

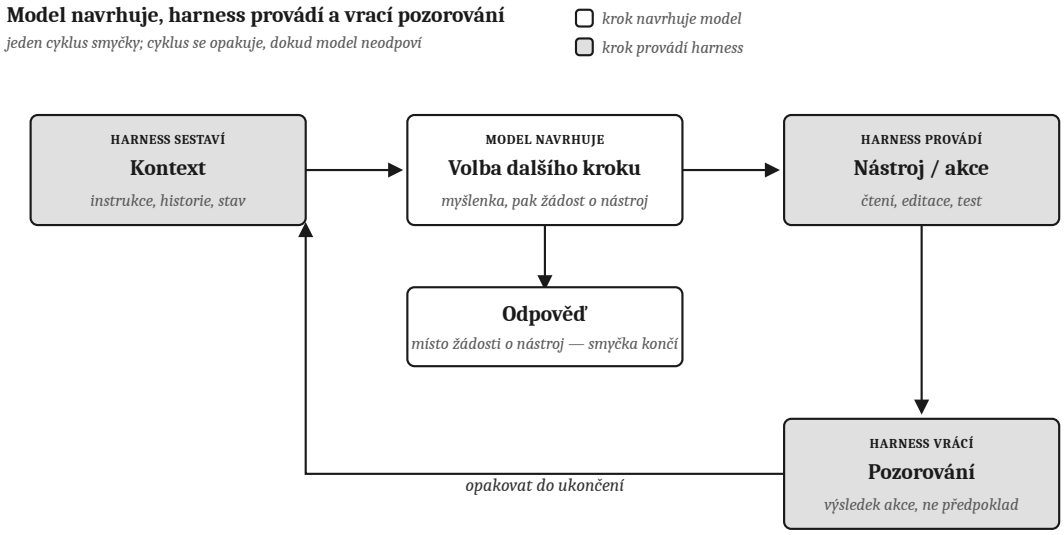
- sestavit kontext — systémové pokyny, dosavadní průběh a stav prostředí
- nechat model odpovědět
- provést požadovaný nástroj
- zapsat výsledek jako pozorování
- vrátit se k dalšímu kroku

Harness provádí akci mimo model a výsledek vrací zpět do kontextu. Smyčka pokračuje, dokud model nevydá závěrečnou odpověď místo dalšího požadavku na nástroj. Průběh shrnuje Obrázek 6.

[6] ***Embedding*** — Číselný vektor, který zachycuje vlastnosti nebo význam objektu tak, aby podobné objekty ležely v prostoru blízko sebe.

[7] ***Agent Loop*** — Opakovaný cyklus, v němž model vyhodnotí stav, zvolí další akci, obdrží její výsledek a pokračuje.

[8] ***ReAct*** — Vzor střídající uvažování modelu, akci nad prostředím a pozorování výsledku.



Obrázek 6 — ***ReAct*** (13).

2.1.3 Nástroje a rozšíření

Zadání určuje, co má agent udělat, zatímco harness určuje, co skutečně může provést (10).

- ***Tools (nástroje)***[9] jsou přímým kanálem mezi modelem a prostředím (14)
- ***Skills (dovednosti)***[10] sjednocují opakovanou práci do znovu použitelných schopností (15)
- ***Hooks***[11] přesouvají vybrané kontroly z úsudku modelu do programu (16)
- ***MCP (Model Context Protocol)***[12] sjednocuje způsob, jakým se tyto externí schopnosti připojují (17)

Každá akce může být navíc řízena oprávněním, které ji povolí automaticky, vyžádá souhlas člověka nebo ji zakáže.

Projektové instrukce lze verzovat přímo s repozitářem. Soubor `AGENTS.md` poskytuje společné místo pro příkazy sestavení, testy a konvence (18). Standard Agent Skills používá pro každou dovednost vstupní soubor `SKILL.md` (15).

[9] ***Tools*** — Funkce zpřístupněné modelu pro práci s prostředím, například čtení souborů, vyhledávání nebo spouštění příkazů.

[10] ***Skills*** — Opakovaně použitelné balíčky instrukcí, skriptů a zdrojů pro určitý typ úlohy.

[11] ***Hooks*** — Programové reakce na události životního cyklu, které mohou před akcí nebo po ní vynutit další krok.

[12] ***MCP (Model Context Protocol)*** — Standard klient–server pro připojování externích nástrojů a datových zdrojů k agentním systémům.

2.1.4 Kontext

Do *__Context Window (kontextové okno)__[13] se v agentním běhu skládají instrukce, části repozitáře, historie nástrojů i výsledky předchozích kroků. Samotná velikost okna nezaručuje správné využití všech informací. Výkon modelů může klesat například tehdy, když se důležitá informace nachází uprostřed dlouhého vstupu (19).

Riziko *__Context Rot (degradace kontextu)__[14] roste s délkou a množstvím méně relevantních informací (20). *__Compaction (kompakce)__[15] uvolňuje místo v kontextovém okně, ale zároveň může zahodit detail, který by byl později užitečný (20).

Dlouhodobý autoritativní stav je proto vhodné uchovávat mimo samotný přepis konverzace a do pracovního kontextu v každém kroku vybírat jen aktuální informace potřebné pro danou úlohu.

[13] __Context Window__ — Množství vstupních a průběžných informací, které může model zpracovat v jednom běhu.

[14] __Context Rot__ — Zhoršování schopnosti modelu využít relevantní informace s rostoucím nebo nekvalitním kontextem.

[15] __Compaction__ — Nahrazení starší části průběhu kratším souhrnem důležitých rozhodnutí, výsledků a otevřených úkolů.

2.2 Agentické inženýrství

Agentic Engineering (agentické inženýrství)[16] přesouvá pozornost od samotného modelu k celému systému, ve kterém agent pracuje (21).

Mezi jeho hlavní oblasti patří ***Prompt Engineering (inženýrství promptů)***[17] (22), ***Context Engineering (kontextové inženýrství)***[18] (20), ***Harness Engineering (inženýrství harnessu)***[19] (23; 24), ***Loop Engineering (inženýrství smyček)***[20] (25) a ***Workflow/Graph Engineering (inženýrství pracovních postupů a grafů)***[21] (26).

2.2.1 Specifikace

Spec-first Development (vývoj od specifikace)[22] navazuje na starší ***Requirements Engineering (inženýrství požadavků)***[23]. IEEE vydalo standard pro Software Requirements Specification už v roce 1984 (27) a současný ISO/IEC/IEEE 29148 formalizuje práci s požadavky v průběhu životního cyklu (28).

U agentů může specifikace přímo vymezit cíl a hranice implementace, zatímco konkrétní technický postup zvolí agent. Oddělení návrhu plánu od jeho provedení ukazuje Obrázek 7.

[16] ***Agentic Engineering*** — Návrh a řízení softwarového vývoje tak, aby agenti pracovali uvnitř explicitně navrženého systému pravidel, kontextu, nástrojů, kontrol a lidských rozhodnutí.

[17] ***Prompt Engineering*** — Formulace instrukcí, omezení, příkladů a očekávaného výstupu konkrétního modelového kroku.

[18] ***Context Engineering*** — Výběr, uspořádání, obnova a kompakce informací dostupných modelu v daném kroku.

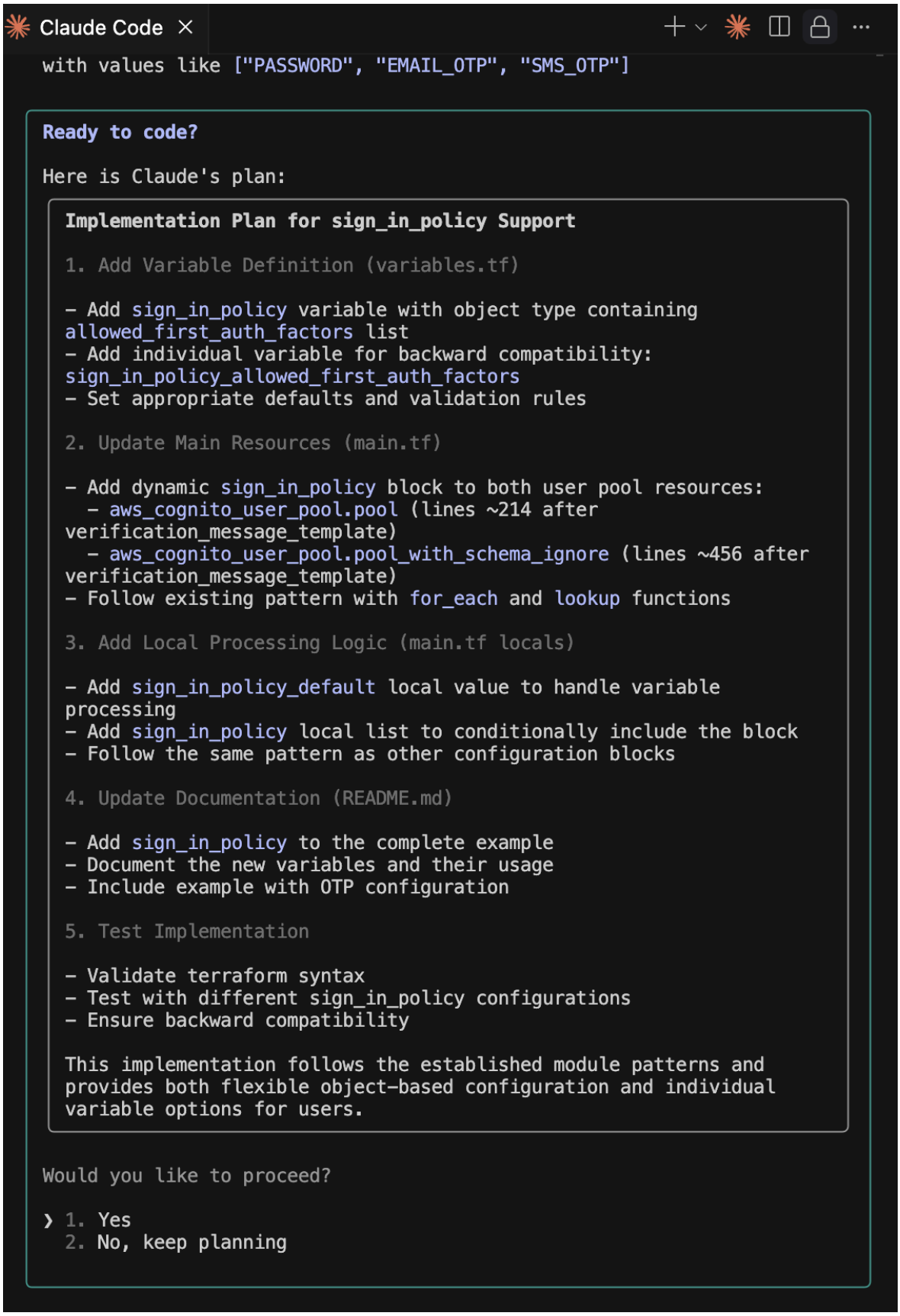
[19] ***Harness Engineering*** — Návrh okolní vrstvy, která modelu poskytuje nástroje, stav, oprávnění a pravidla běhu.

[20] ***Loop Engineering*** — Návrh řídicích smyček, které opakovaně porovnávají stav s cílem a rozhodují o dalším kroku.

[21] ***Workflow/Graph Engineering*** — Disciplína zaměřená na návrh pořadí, závislostí a větvení mezi více kroky nebo agenty.

[22] ***Spec-first Development*** — Postup, v němž je požadovaný výsledek a jeho omezení popsán před samotnou implementací.

[23] ***Requirements Engineering*** — Systematická práce se zjišťováním, popisem a správou požadavků na software.

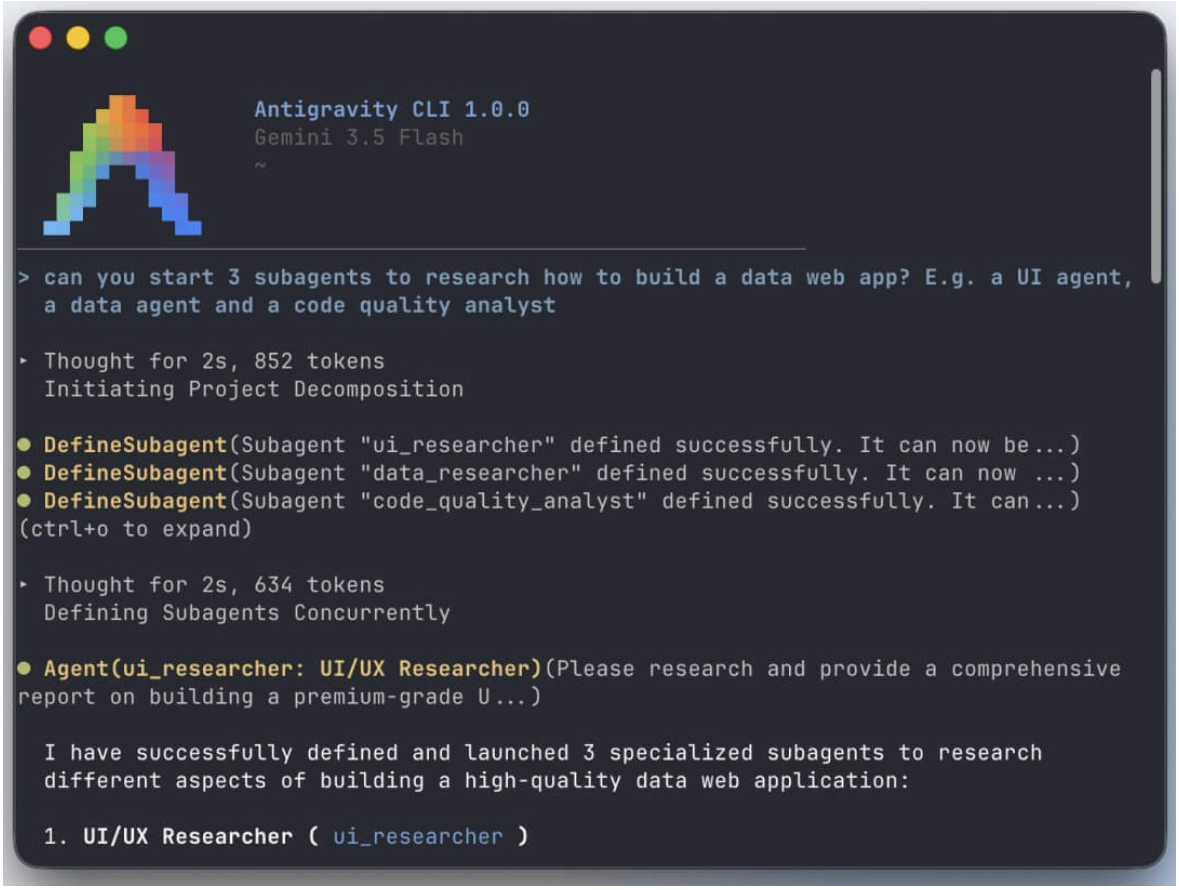


Obrázek 7 — Plán před implementací v Claude Code (29).

2.2.2 Orchestrace

Orchestration (orchestrace)[24] se stává důležitou tam, kde úloha přesahuje jeden agentní běh. Více agentů samo o sobě nezaručuje lepší výsledek. Přínos vzniká až tehdy, když jsou jejich úlohy vhodně rozdělené a výstupy znovu spojeny vůči společnému cíli.

- ***Coordinator/Subagent (koordinátor a subagent)***[25] umožňuje paralelizovat nezávislé části práce Obrázek 8



Obrázek 8 — Paralelní subagenti v Google Antigravity (30).

- ***Workflow Graph (graf pracovního postupu)***[26] lze vykonávat programově mimo kontext modelu. Claude Code může u dynamických pracovních postupů vytvořit skript, který takový graf realizuje (31) Obrázek 9

[24] ***Orchestration*** — Řízení pořadí, závislostí, sdíleného stavu a integračních kontrol mezi více kroky nebo agenty.

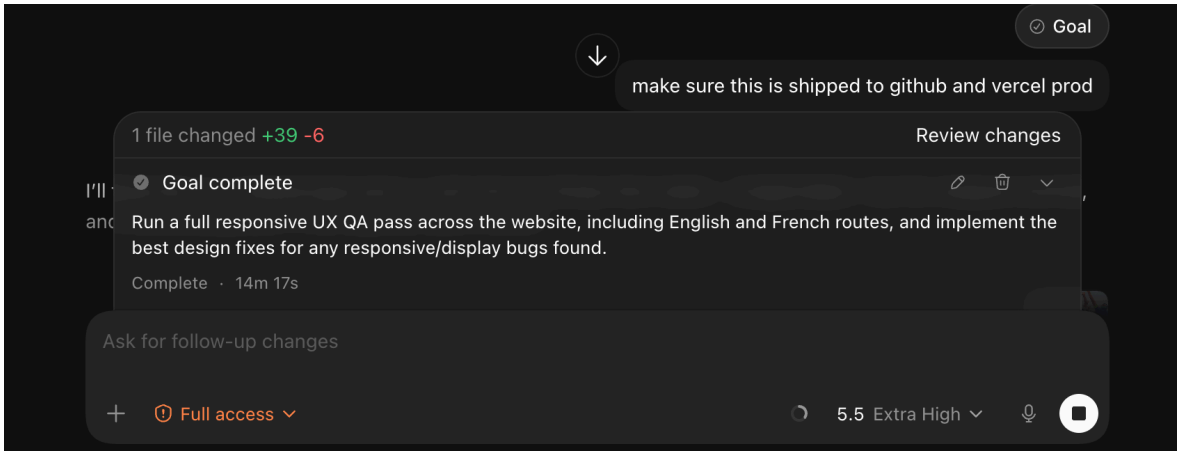
[25] ***Coordinator/Subagent*** — Vzor, v němž koordinátor deleguje dílčí úlohy samostatným agentům s vlastním kontextem.

[26] ***Workflow Graph*** — Graf, který předem určuje závislosti, pořadí a větvení kroků.



Obrázek 9 — Dynamický pracovní postup v Claude Code (31).

- ***Goal Loop (cílová smyčka)***[27] se v jednoduché podobě objevuje ve vzoru ***Ralph Loop (Ralphova smyčka)***[28] popsaném Geoffreyem Huntleym (32). Podobné cílové smyčky dnes podporují Claude Code i Codex (25; 33) Obrázek 10



Obrázek 10 — Cílová smyčka v Codexu (25).

- ***Human-in-the-loop (člověk v rozhodovací smyčce)***[29] se může projevit jako schválení specifikace, plánu nebo výsledné změny

[27] ***Goal Loop*** — Nadřazená smyčka, která po dílčím běhu porovná stav s cílem a rozhodne, zda práce skončí nebo pokračuje.

[28] ***Ralph Loop*** — Jednoduchý vzor opakovaného spouštění coding agenta, který drží stav v pracovním stromu místo v přepisu konverzace.

[29] ***Human-in-the-loop*** — Zapojení explicitního lidského rozhodnutí do jinak automatizovaného procesu.

2.3 Softwarová továrna

Pojem ***Software Factory (softwarová továrna)***[30] není nový. Robert W. Bemer jej už na konferenci NATO o softwarovém inženýrství v roce 1968 popsal jako *machine-controlled production environment, or software factory* (34). Program, jeho kontrola i používání měly probíhat uvnitř společného prostředí vybaveného nástroji pro práci se soubory, kompilaci, testování a sestavení systému.

Tradiční automatizace funguje nejlépe tam, kde lze postup předem přesně popsat programem. ***Coding Agents (coding agenti)*** tuto hranici posouvají, protože model může podle aktuálního stavu volit další kroky a používat nástroje i tehdy, když celý postup není dopředu pevně určen. Softwarová továrna tak může spojit deterministickou automatizaci, agentní kroky a lidské rozhodování.

Takové uspořádání se již používá v produkčním softwarovém inženýrství. Stripe uvádí, že jeho interní agenti Minions pravidelně vytvářejí požadavky na sloučení určené k lidské revizi (7). Meta popisuje vlastní agentní platformu, která automatizuje hledání a opravu výkonnostních problémů a dovádí proces až k požadavku na sloučení připravenému pro člověka (8).

DarkFactory představuje záměrně jednoduchý počáteční základ tohoto přístupu. První implementace už dokáže agenty spouštět a řídit v repozitářovém procesu a současně vytváří základ pro další rozvoj stejným způsobem.

[30] ***Software Factory*** — Vývojové prostředí, které převádí tvorbu softwaru do co nejvíce řízeného, opakovatelného a nástroji podporovaného procesu.

PRAKTICKÁ ČÁST

DarkFactory

3 Praktická část

3.1 Metodika

Praktická část má podobu inženýrské případové studie systému DarkFactory, konkrétní realizace širšího vzoru softwarové továrny nativně postavená na GitHubu. Cílem není měřit obecnou úspěšnost jazykových modelů ani porovnávat konkrétní agenty, ale na jednom reálném systému ukázat, jak se principy popsané v teoretické části propojují v produkčním vývojovém procesu.

Popis implementace a následná zjištění se vztahují ke konkrétní revizi DarkFactory `d576ec8f` (35). Analýza vychází ze zdrojového kódu, pracovních postupů GitHub Actions, řídicího skriptu, registru harnessů, projektových pravidel a dokumentační konfigurace této revize.

Postup případové studie navazuje na teoretickou část. Nejprve jsou vymezeny opakující se principy agentického vývoje a následně je sledováno, jak jsou realizovány v DarkFactory. Z implementace je rekonstruován celý průchod požadavku systémem a diskuse potom odděleně shrnuje zjištění a omezení konkrétní revize.

DarkFactory byla implementována pomocí komerčních ***Coding Agents (coding agenti)***. Počáteční podoba je záměrně jednoduchá a slouží jako počáteční základ, který již dokáže stejné agenty spouštět a řídit. Praktická část proto sleduje nejen výslednou architekturu, ale také způsob, jakým se v jednom procesu doplňují agent, harness, programová orchestrace a člověk.

3.2 Návrh systému DarkFactory

DarkFactory byla navržena jako záměrně malá realizace principů popsaných v teoretické části. Cílem první verze nebylo pokrýt co nejvíce funkcí, ale uzavřít nejmenší prakticky použitelný vývojový cyklus. Systém přijme požadavek, předá otevřenou inženýrskou práci agentovi, zachová průběžný stav a vrátí výsledek do běžného procesu na GitHubu. Rozsah byl zvolen tak, aby bylo možné první funkční podobu vytvořit v jednom souvislém agentním běhu a další změny už provádět stejným procesem.

GitHub slouží nejen jako hosting repozitáře, ale také jako hlavní rozhraní a trvalá stavová vrstva systému. Požadavek, diskuse, schválení, větev, commity a požadavek na sloučení proto zůstávají mimo kontext modelu a přežívají jednotlivé agentní běhy. Události GitHubu spouštějí příslušné pracovní postupy GitHub Actions, takže DarkFactory pro základní průchod nepotřebuje vlastní trvale běžící server. Každý agentní krok probíhá odděleně a pracovní prostředí je izolováno kontejnerem (35).

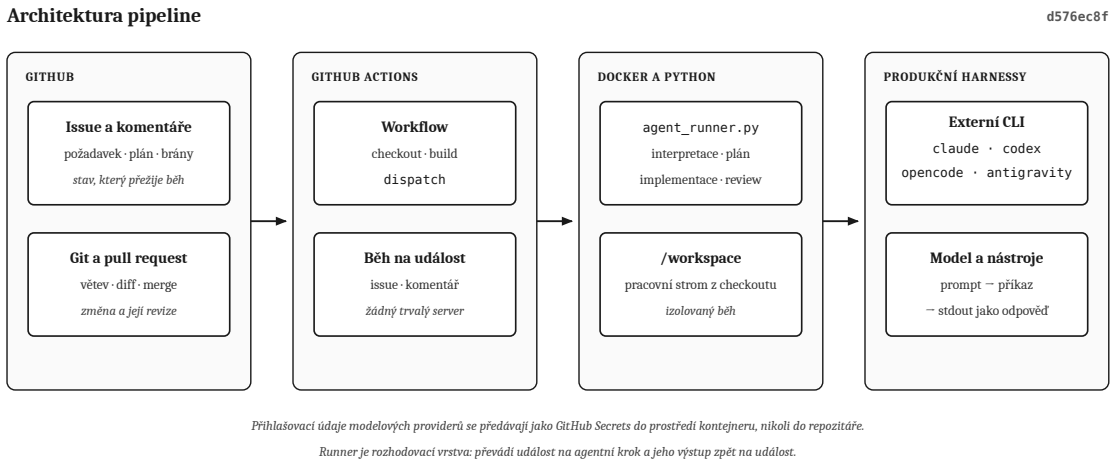
DarkFactory neimplementuje vlastní agentní běhové prostředí. Produkční ***Harness*** zajišťuje agentní smyčku, komunikaci s modelem, nástroje a práci s kontextem. DarkFactory nad tím řídí pořadí kroků, stav procesu a lidské brány. Otevřené úlohy řeší model prostřednictvím harnessu, zatímco přesně určitelné kroky, například práce s větví, testy, formátování nebo změna stavu, zůstávají programově řízené (35).

Dokumentace tvoří rozhraní mezi automatizovaným procesem a člověkem. Ve zkoumané revizi vzniká dokumentace veřejných rozhraní přímo u zdrojového kódu a dokumentační web se generuje z kanonických souborů repozitáře. Publikovaná dokumentace tak není druhou ručně udržovanou kopií systému, ale čitelným pohledem nad stejnými zdroji, se kterými pracují agenti a automatizace (35).

Člověk nemusí řídit jednotlivé technické kroky, ale schvaluje záměr a plán před implementací a na konci rozhoduje o přijetí výsledku. Mezi těmito body proces pokračuje automaticky. První verze tak současně tvoří počáteční základ pro další vývoj DarkFactory. Jakmile základní cyklus funguje, může být stejný proces použit i pro další rozšiřování vlastního systému.

3.3 Architektura systému

Pipeline je tvořena GitHub Actions workflow a pythonovskými skripty, které řídí jednotlivé fáze a předávají práci produkčnímu harnessu. Architektura tak rozděluje odpovědnost mezi GitHub, GitHub Actions, izolované pracovní prostředí a harness. GitHub uchovává stav, Actions spouštějí jednotlivé fáze, Docker odděluje agentní běh a řídicí skript převádí stav procesu na konkrétní agentní krok. Samotnou agentní smyčku, model a nástroje zajišťuje zvolený harness (35). Přehled ukazuje Obrázek 11.



Obrázek 11 — Architektura DarkFactory (35).

GitHub uchovává uživatelský požadavek, interpretaci a diskusi. Pracovní větev a commity nesou implementaci a požadavek na sloučení slouží k automatické i lidské revizi. Stav proto není závislý na jednom kontextovém okně modelu (35).

GitHub Actions reagují na otevření požadavku, nový komentář, komentář k revizi, interní událost nebo ruční spuštění. Každý běh připraví repozitář a spustí agenta v kontejneru. DarkFactory tak používá Actions jako událostní mechanismus i krátkodobé výpočetní prostředí bez vlastní trvale běžící služby (35).

Soubor `agent_runner.py` řídí fáze interpretace, plánování, implementace, automatické revize, kontroly souladu s plánem a reakce na zpětnou vazbu. Vlastní agentní práce je oddělena pomocí registru v `harnesses.py`, který popisuje podporované harnessy, jejich příkazové rozhraní, autentizaci a záložní modely (35).

Přihlašovací údaje jsou od trvalého stavu oddělené. Operace nad GitHubem používají autorizovaný token nebo GitHub App, zatímco přihlašovací údaje poskytovatelů modelů vstupují pouze do běhu, který je potřebuje (35).

3.4 Průchod požadavku systémem

Průchod začíná vytvořením požadavku s uživatelským zadáním. Repozitář nabízí předlohu se šesti poli, z nichž tři jsou povinná (35). Strukturu shrnuje Tabulka 1.

Pole předlohy požadavku	
`Verbatim User Request`	povinné
`Area / Component`	povinné
`Request Type`	povinné
`Parent Epic`	volitelné
`Proposed Acceptance Criteria`	volitelné
`Additional Context`	volitelné

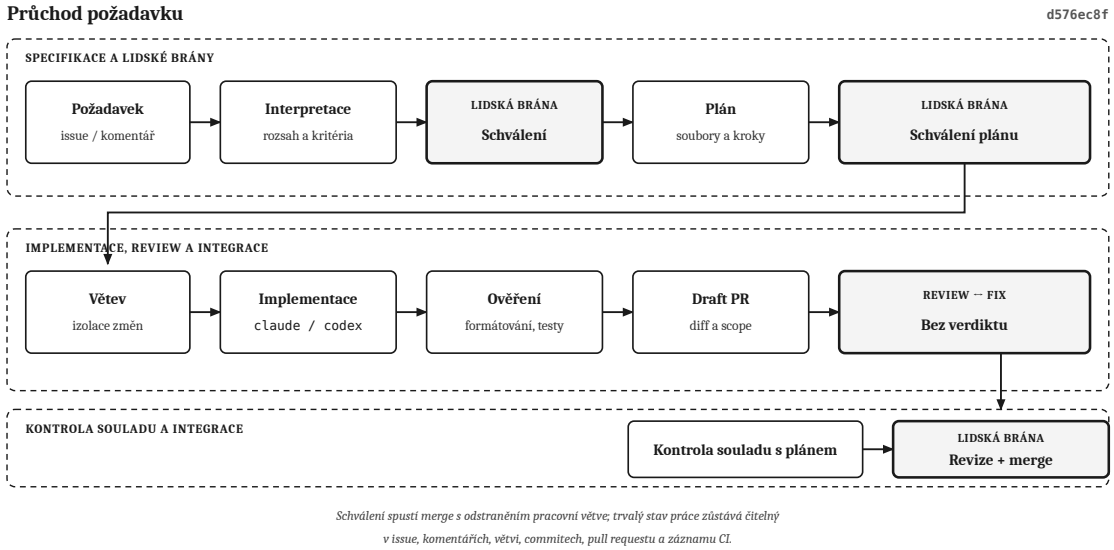
Tabulka 1 — Pole předlohy požadavku (35).

Řídicí skript předá zadání modelu k interpretaci a výsledek zapíše jako komentář. Po lidském schválení vznikne propojený plánovací požadavek `Plan`, ve kterém agent připraví implementační plán. Také plán musí člověk schválit před vznikem pracovní větve (35).

Po schválení plánu vznikne nebo se obnoví pracovní větev. Agent provede změny v izolovaném prostředí a řídicí skript následně spustí dostupné formátovací a testovací příkazy. Při neúspěchu dostane agent jeden opravný krok. Poté se změna uloží do commitu, větev se odešle a otevře se koncept požadavku na sloučení (35).

Nad požadavkem na sloučení pokračuje automatická revizní smyčka. Model hledá konkrétní problémy a nález vrací do opravného běhu. Současně se nad větví spouštějí automatické integrační kontroly. Smyčka je omezena na tři iterace. Pokud ani poslední iterace neskončí čistým verdiktem, proces pokračuje ke kontrole souladu s plánem (35).

Samostatný modelový krok následně posoudí, zda výsledná změna odpovídá schválenému plánu. Po úspěšném dokončení této kontroly může být změna předána člověku. Celý průchod shrnuje Obrázek 12.



Obrázek 12 — Průchod požadavku DarkFactory (35).

Člověk může změnu schválit nebo vrátit zpětnou vazbu. Požadavek na úpravu spustí další opravný běh na stejné větvi. Konečné schválení zpracuje samostatný pracovní postup, který po splnění podmínek provede sloučení a odstraní pracovní větev (35).

Celý proces tak střídá modelový úsudek, deterministickou automatizaci a explicitní lidská rozhodnutí.

3.5 Implementace pomocí agentů

DarkFactory byla sama vyvíjena pomocí komerčních ***Coding Agents (coding agenti)***. Člověk určoval cíl a zásadní rozhodnutí, zatímco samotná implementace proběhla v jednom souvislém agentním běhu.

Běh nezačal psaním kódu, ale plánovacím režimem. Prvním cílem agenta bylo navrhnout architekturu DarkFactory a rozdělit odpovědnost mezi GitHub, automatizaci, Harness a lidské rozhodovací body. V této fázi vznikl plán systému ještě před vlastní implementací.

Po vymezení architektury dostal stejný agent roli orchestrátora. Místo toho, aby celý systém implementoval sekvenčně sám, rozděloval dílčí práci mezi subagenty, přebíral jejich výsledky a skládal je zpět vůči společnému cíli. Praktická implementace tak přímo použila princip koordinátora a subagentů popsaný v teoretické části.

Počáteční rozsah byl záměrně omezen na nejmenší uzavřený proces, který dokáže přijmout požadavek, předat práci agentovi a vrátit výsledek do řízeného procesu na GitHubu. Smyslem bylo co nejrychleji dosáhnout funkčního základu, který už může podporovat vlastní další vývoj.

Před vznikem této základní verze musel člověk agenta spouštět a řídit přímo. Po dokončení základního cyklu už DarkFactory dokáže stejné nástroje spouštět sama nad vlastním repozitářem (35). Další funkce tak mohou vznikat postupně prostřednictvím stejného procesu.

Tento způsob vzniku sám o sobě nedokazuje vyšší kvalitu agentem vytvořeného softwaru. Ukazuje však, že principy plánování, orchestrace a delegace použité uvnitř DarkFactory byly použity už při vzniku její první funkční podoby.

4 Zjištění a diskuse

Výzkumná otázka se ptá, jaké principy se opakují v současném agentickém vývoji softwaru a jak je realizuje DarkFactory. Z analyzovaných zdrojů vystupuje společný vzor. Model pracuje uvnitř harnessu a širšího procesu, trvalý stav zůstává mimo jeho kontext, otevřený úsudek se odděluje od programově proveditelných kontrol a důležitá rozhodnutí zůstávají člověku (10; 20; 26).

DarkFactory tento vzor realizuje pomocí běžného repozitářového procesu. GitHub uchovává stav práce, automatizace řídí pořadí kroků a agent řeší úlohy, které nelze předem přesně naprogramovat (35). Výsledek se vrací do větve a požadavku na sloučení, takže agentní práce zůstává čitelná a kontrolovatelná stejnými nástroji jako práce člověka. Podobnou hranici mezi autonomní prací a lidskou revizí popisují také Stripe a Meta (7; 8).

Prakticky důležitá je i dokumentace generovaná z kanonických zdrojů. Agent pracuje s verzovanými pravidly a kódem, zatímco člověk dostává čitelný pohled nad stejným systémem. Dokumentace tak propojuje automatizovanou implementaci s lidskou kontrolou (35).

Počáteční implementace má současně konkrétní omezení. Modelová revize není deterministickým důkazem správnosti, omezená revizní smyčka může skončit bez čistého verdiktu, verifikace se po jednom opravném kroku znovu neopakuje a změna rozsahu může pokračovat bez obnoveného schválení původního plánu (35). Zjištění proto nelze zobecnit na všechny modely nebo repozitáře.

Případ DarkFactory ukazuje především to, že praktický agentický vývoj nevzniká samotným modelem. Vzniká až spojením specifikace, kontextu, nástrojů, trvalého stavu, orchestrace, kontrol a lidského rozhodování. Záměrně malá první verze navíc ukazuje, že takový proces lze použít jako počáteční základ pro další rozvoj stejného systému.

5 Závěr

Cílem práce bylo popsat a systematizovat principy současného agentického inženýrství a na systému DarkFactory ukázat jejich praktické propojení. Tento cíl byl naplněn propojením teoretického rozboru s případovou studií konkrétní implementace.

Hlavním závěrem je, že schopnost modelu sama nestačí. Praktický agentický vývoj závisí také na kvalitě specifikace, kontextu, nástrojů, trvalého stavu, orchestrace, programových kontrol a lidských rozhodovacích bodů. DarkFactory ukazuje jednu konkrétní podobu tohoto spojení a zároveň to, že malá počáteční implementace může sloužit jako základ pro další rozvoj pomocí stejného procesu.

Výsledek je omezen na jeden systém a jednu jeho revizi. Další práce by proto měla zpřesnit deterministické brány, znovu vyžadovat schválení při změně rozsahu a ověřit stejné principy na dalších repozitářích a s dalšími harnessy.

Agentické inženýrství tak přesouvá část práce vývojáře od ručního provádění jednotlivých kroků k návrhu prostředí, ve kterém agent ví, co má udělat, co smí udělat, jak se jeho výstup ověří a kdy musí rozhodnout člověk.

Seznam zdrojů

1. GITHUB. *GitHub Copilot code suggestions in your IDE* [online]. 2026 [vid. 2026-09-22]. Dostupné z: https://docs.github.com/en/copilot/concepts/completions/code-suggestions

2. KHURANA, Sonam a Anusha VADDI. ChatGPT From the Perspective of an Academic Oral and Maxillofacial Radiologist. *Cureus* [online]. 2023, **15**(6). Dostupné z: doi:10.7759/cureus.40053. Obrázek 4. Screenshot z bezplatné verze ChatGPT, převzato jako ilustrace

3. GITHUB. *Use GitHub Copilot agents* [online]. 2026 [vid. 2026-09-22]. Dostupné z: https://docs.github.com/en/copilot/how-tos/use-copilot-agents

4. OPENAI. *Introducing Codex* [online]. 2025 [vid. 2026-09-22]. Dostupné z: https://openai.com/index/introducing-codex/

5. OPENAI. *Introducing the Codex app* [online]. 2026 [vid. 2026-09-22]. Dostupné z: https://openai.com/index/introducing-the-codex-app/

6. HILLEBRANDT, Finn. *How Many People Use AI? Facts & Figures 2026* [online]. 2026 [vid. 2026-09-22]. Dostupné z: https://www.gradually.ai/en/how-many-people-use-ai/. Published 2026-08-23

7. STRIPE. *Developer keynote: Advancing developer craft* [online]. 2026 [vid. 2026-09-30]. Dostupné z: https://stripe.com/sessions/2026/developer-keynote. Stripe Sessions 2026

8. META. *Capacity Efficiency at Meta: How Unified AI Agents Optimize Performance at Hyperscale* [online]. 2026 [vid. 2026-09-30]. Dostupné z: https://engineering.fb.com/2026/04/16/developer-tools/capacity-efficiency-at-meta-how-unified-ai-agents-optimize-performance-at-hyperscale/

9. MERRIAM-WEBSTER. *Agentic: Definition & Meaning* [online]. 2026 [vid. 2026-09-26]. Dostupné z: https://www.merriam-webster.com/dictionary/agentic. Původ slova: agent + -ic, poprvé doloženo 1966

10. TRIVEDY, Vivek. *The Anatomy of an Agent Harness* [online]. 2026 [vid. 2026-09-27]. Dostupné z: https://www.langchain.com/blog/the-anatomy-of-an-agent-harness. LangChain, 10. března 2026

11. VASWANI, Ashish, Noam SHAZEER, Niki PARMAR, Jakob USZKOREIT, Llion JONES, Aidan N. GOMEZ, Łukasz KAISER a Illia POLOSUKHIN. Attention Is All You Need.

arXiv preprint arXiv:1706.03762 [online]. 2017. Dostupné z: https://arxiv.org/abs/1706.03762

12. MIKOLOV, Tomas, Wen-tau YIH a Geoffrey ZWEIG. Linguistic Regularities in Continuous Space Word Representations. In: *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* [online]. 2013, s. 746–751. Dostupné z: https://aclanthology.org/N13-1090/

13. YAO, Shunyu, Jeffrey ZHAO, Dian YU, Nan DU, Izhak SHAFRAN, Karthik NARASIMHAN a Yuan CAO. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629* [online]. 2022. Dostupné z: https://arxiv.org/abs/2210.03629

14. ANTHROPIC. *Building effective agents* [online]. 2024 [vid. 2026-09-23]. Dostupné z: https://www.anthropic.com/engineering/building-effective-agents

15. AGENT SKILLS. *Agent Skills: an open format for extending AI agents* [online]. 2025 [vid. 2026-09-22]. Dostupné z: https://agentskills.io/home

16. OPENAI. *Lifecycle | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-22]. Dostupné z: https://openai.github.io/openai-agents-python/ref/lifecycle/

17. MODEL CONTEXT PROTOCOL. *Model Context Protocol 2026-07-28 specification: server features* [online]. 2026 [vid. 2026-09-22]. Dostupné z: https://modelcontextprotocol.io/specification/2026-07-28/server

18. AGENTIC AI FOUNDATION. *AGENTS.md* [online]. 2026 [vid. 2026-09-23]. Dostupné z: https://agents.md/

19. LIU, Nelson F., Kevin LIN, John HEWITT, Ashwin PARANJAPE, Michele BEVILACQUA, Fabio PETRONI a Percy LIANG. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* [online]. 2024, **12**, 157–173. Dostupné z: https://arxiv.org/abs/2307.03172

20. ANTHROPIC. *Effective context engineering for AI agents* [online]. 2025 [vid. 2026-09-22]. Dostupné z: https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

21. ALENEZI, Mamdouh. Educating the Agentic Engineer: Curricula, Collaboration, and Continuous Learning in the AI Era. *arXiv preprint arXiv:2607.29610* [online]. 2026 [vid. 2026-09-26]. Dostupné z: https://arxiv.org/abs/2607.29610

22. OPENAI. *Prompt engineering* [online]. 2026 [vid. 2026-09-22]. Dostupné z: https://platform.openai.com/docs/guides/prompt-engineering

23. ANTHROPIC. *Harness design for long-running application development* [online]. 2026 [vid. 2026-09-21]. Dostupné z: https://www.anthropic.com/engineering/harness--design-long-running-apps

24. OPENAI. *Sandbox agents: Concepts | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-22]. Dostupné z: https://openai.github.io/openai-agents-js/guides/sandbox-agents/concepts/

25. OPENAI. *Using Goals in Codex: Persistent Objectives for Long-Running Work* [online]. 2026 [vid. 2026-09-27]. Dostupné z: https://developers.openai.com/cookbook/examples/codex/using_goals_in_codex

26. OPENAI. *Agent orchestration | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-21]. Dostupné z: https://openai.github.io/openai-agents-python/multi_agent/

27. IEEE. *IEEE Guide for Software Requirements Specifications* [online]. 1984 [vid. 2026-09-30]. Dostupné z: https://standards.ieee.org/ieee/830/1220/. IEEE 830-1984

28. ISO/IEC/IEEE. *ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering* [online]. 2018 [vid. 2026-09-30]. Dostupné z: https://www.iso.org/standard/72089.html

29. GALLARDO, Luis. *Beyond Cursor: My Experience with Claude Code and Sequential Thinking* [online]. 2025 [vid. 2026-09-28]. Dostupné z: https://lgallardo.com/2025/07/09/beyond-cursor-claude-code-sequential-thinking/. Obrázek s plánem, který Claude Code předkládá ke schválení. Převzatá ilustrace rozhraní, nikoli záznam běhu popsané pipeline

30. GOOGLE. *Antigravity CLI* [online]. 2026 [vid. 2026-09-27]. Dostupné z: https://antigravity.google/product/antigravity-cli. Sekce Subagents functionalities. Převzatá ilustrace rozhraní, nikoli záznam běhu

31. ANTHROPIC. *Orchestrate subagents at scale with dynamic workflows* [online]. 2026 [vid. 2026-09-27]. Dostupné z: https://code.claude.com/docs/en/workflows. Sekce Dynamic workflows. Převzatá ilustrace rozhraní, nikoli záznam běhu

32. HUNTLEY, Geoffrey. *Ralph Wiggum as a „software engineer“* [online]. 2025 [vid. 2026-09-27]. Dostupné z: https://ghuntley.com/ralph/. 14. července 2025. Původní text, který techniku pojmenoval

33. ANTHROPIC. *Keep Claude working toward a goal* [online]. 2026 [vid. 2026-09-27]. Dostupné z: https://code.claude.com/docs/en/goal

34. NATO. *Software Engineering: Report of a Conference Sponsored by the NATO Science Committee, Garmisch, Germany, 7–11 October 1968* [online]. Brussels: NATO, Scientific Affairs Division, 1969 [vid. 2026-09-27]. Dostupné z: http://homepages.cs.ncl.ac.uk/brian.randell/NATO/nato1968.PDF. 231 s. Zpráva o softwarové krizi; stažitelný přepis z roku 2001 má 136 stran, číslování citací se vztahuje k původnímu vydání

35. MARIUS, Patrik. *DarkFactory revize d576ec8f* [online]. 2026 [vid. 2026-09-26]. Dostupné z: https://github.com/marius-patrik/DarkFactory/tree/d576ec8fdb78a654bfffc9d0da165aa81a97b673. Poslední revize, v níž ještě pipeline volá produkční harnessy

Seznam součástí textu

Obrázek 1 Doplňování kódu v editoru (1). 6
Obrázek 2 Chatbot bez přístupu k obrázku (2). 7
Obrázek 3 Odhad využití generativní AI podle typu (6). 8
Obrázek 4 Vrstvy agentického systému. 11
Obrázek 5 Sémantické vztahy ve vektorovém prostoru (12). 12
Obrázek 6 ***ReAct*** (13). 13
Obrázek 7 Plán před implementací v Claude Code (29). 16
Obrázek 8 Paralelní subagenti v Google Antigravity (30). 17
Obrázek 9 Dynamický pracovní postup v Claude Code (31). 18
Obrázek 10 Cílová smyčka v Codexu (25). 18
Obrázek 11 Architektura DarkFactory (35). 23
Tabulka 1 Pole předlohy požadavku (35). 24
Obrázek 12 Průchod požadavku DarkFactory (35). 25